

24UCS171 - Programming in C

Mini Project: Transaction Processing System

Total Marks: 80

Pattern: 10 Questions × 8 Marks

(2 Questions for Each Course Outcome)

Q1 a) Program Compilation and Execution (8 Marks)

Tasks

1. Compile and execute the given `trans.c` program using Replit/GCC.
2. Demonstrate all menu-driven operations with sample inputs.
3. Explain the use of:
 - variables
 - loops
 - conditional statements
 - switch-case

Evaluation

Component	Marks
Successful compilation and execution	2
Demonstration of menu operations	2
Explanation of control structures	2
Sample testing and output	2

Q1 b) Program Analysis and Debugging (8 Marks)

Tasks

1. Test the existing program using different account transactions.
2. Identify at least two logical/runtime issues.
3. Correct the identified issues and justify the corrections.

Evaluation

Component	Marks
Testing effort	2
Identification of issues	3
Corrected logic and explanation	3

Q2 a) Searching using Arrays and Strings (8 Marks)

Tasks

1. Store and manage account records using arrays/strings.
2. Implement searching based on:
 - account number OR
 - customer name
3. Display matching account information.

Evaluation

Component	Marks
Proper use of arrays/strings	3
Searching implementation	3
Output correctness	2

Q2 b) Sorting Account Records (8 Marks)

Tasks

1. Implement any one sorting algorithm.
2. Sort records based on:
 - account number OR
 - balance amount
3. Display sorted records.

Evaluation

Component	Marks
Sorting logic	3
Correct implementation	3
Display and testing	2

Q3 a) Functional Decomposition (8 Marks)

Tasks

1. Refactor the program into separate functions for:
 - user input
 - transaction processing
 - display operations
2. Improve readability and modularity.

Evaluation

Component	Marks
Function decomposition	4
Modular design and readability	4

Q3 b) Pointer-Based Operations (8 Marks)

Tasks

1. Use pointers for:
 - structure access
 - balance updates
 - function parameter passing
2. Explain advantages of pointer usage in the program.

Evaluation

Component	Marks
Proper pointer implementation	4

Component	Marks
Explanation and correctness	4

Q4 a) Structure Enhancement (8 Marks)

Tasks

1. Analyze the existing account structure.
2. Add any two additional fields:
 - mobile number
 - account type
 - ATM card number
 - minimum balance
 - transaction count

Evaluation

Component	Marks
Structure modification	4
Proper implementation and testing	4

Q4 b) New Banking Feature Implementation (8 Marks)

Tasks

Implement any one new banking feature:

- ATM debit transaction
- negative balance restriction
- mini statement
- balance transfer
- account listing in tabular format

Evaluation

Component	Marks
Feature implementation	4
Functionality and innovation	4

Q5 a) File Generation and Verification (8 Marks)

Tasks

1. Generate accounts.txt file.
2. Add new account details and regenerate the file.
3. Verify whether file contents are correctly updated.
4. Correct any file-writing issues identified.

Evaluation

Component	Marks
File generation	2
File update verification	3
Correction of file issues	3

Q5 b) Optimization and Error Handling (8 Marks)

Tasks

1. Improve program efficiency for:
 - memory usage
 - processing speed
2. Add error handling for:
 - invalid account number
 - insufficient balance
 - file opening failure

Evaluation

Component	Marks
Optimization techniques	4
Error handling implementation	4

Final Mark Distribution

Course Outcome	Questions	Marks
CO1	Q1A, Q1B	16
CO2	Q2A, Q2B	16
CO3	Q3A, Q3B	16
CO4	Q4A, Q4B	16
CO5	Q5A, Q5B	16
Total	10 Questions	80 Marks

Internal Assessment (IA) Mark Split

IA1 – Web Portal Entry 1 (40 Marks)

Questions Included	CO Mapping	Marks
Q1A	CO1	8
Q1B	CO1	8
Q2A	CO2	8
Q2B	CO2	8
Q3A	CO3	8
Total		40 Marks

Coverage in IA1

- Basic C programming constructs
- Program execution and debugging
- Arrays and strings
- Searching and sorting
- Functional decomposition

IA2 – Web Portal Entry 2 (40 Marks)

Questions Included	CO Mapping	Marks
Q3B	CO3	8
Q4A	CO4	8
Q4B	CO4	8
Q5A	CO5	8
Q5B	CO5	8
Total		40 Marks

Coverage in IA2

- Pointer-based programming
- Structures and unions
- Banking feature implementation
- File handling concepts
- Optimization and error handling

Additional Instructions

1. Individual work only. Code copying or sharing of source code is strictly prohibited. Any malpractice identified during evaluation will result in disciplinary action and a mark reduction.
2. Every student must maintain an individual GitHub repository for the mini project throughout the development process. The project must be started by forking the base repository: <https://github.com/24UCS271-MiniProject/miniProjectSourceCode>. Students are required to improve from this base code.
3. Students must submit the GitHub repository link within the stipulated deadline.
Repository should contain:
 - source code
 - documentation
 - screenshots/output
 - commit history
 - README file with execution steps
4. Students are expected to perform regular commits showing progressive development of the project. Repositories with single-day bulk uploads may be treated as incomplete development evidence.

5. Repository links will be collected and evaluated using LLM-assisted assessment, along with faculty review based on the prescribed rubrics and Course Outcome (CO) mapped questions.
6. Evaluation will be carried out question-wise according to:
 - functionality
 - coding standards
 - modularity
 - innovation
 - optimization
 - documentation
 - testing evidence
7. Students must demonstrate the project during evaluation using Replit/GitHub/local execution environment.
8. Proper coding standards, naming conventions, indentation, comments, and formatting practices should be followed throughout the implementation.
9. Viva questions may be asked from any part of the submitted code, logic, optimization techniques, or implemented features.
10. Students are encouraged to improve the project creatively beyond the minimum requirements by adding innovative banking functionalities, better user interfaces, optimized processing, or advanced error handling techniques.